

[기술공통]

데이터 엔지니어링



- I. DE 개요
- II. 자료구조와 파이썬 문법
- III. Numpy와 Pandas

I. DE 개요 및 실습환경 이해

• Scaling Laws (Open AI, 2020)

- 컴퓨팅 리소스, 데이터, 모형 크기를 늘릴 수록 성능 개선
- 새로운 능력이 생겨남 (Emergent Abilities)

Scaling Laws for Neural Language Models			
Jared Kaplan *		Sam McCandlish *	
Johns Hopkins University, OpenAI		OpenAI	
jaredk@jhu.edu		sam@openai.com	
Tom Henighan	Tom B. Brown	Benjamin Chess	Rewon Child
OpenAI	OpenAI	OpenAI	OpenAI
henighan@openai.com	tom@openai.com	bchess@openai.com	rewon@openai.com
Scott Gray	Alec Radford	Jeffrey Wu	Dario Amodei
OpenAI	OpenAI	OpenAI	OpenAI
scott@openai.com	alec@openai.com	jeffwu@openai.com	damodei@openai.com

Abstract

We study empirical scaling laws for language model performance on the cross-entropy loss. The loss scales as a power-law with model size, dataset size, and the amount of compute used for training, with some trends spanning more than seven orders of magnitude. Other architectural details such as network width or depth have minimal effects within a wide range. Simple equations govern the dependence of overfitting on model/dataset size and the dependence of training speed on model size. These relationships allow us to determine the optimal allocation of a fixed compute budget. Larger models are significantly more sample-efficient, such that optimally compute-efficient training involves training very large models on a relatively modest amount of data and stopping significantly before convergence.

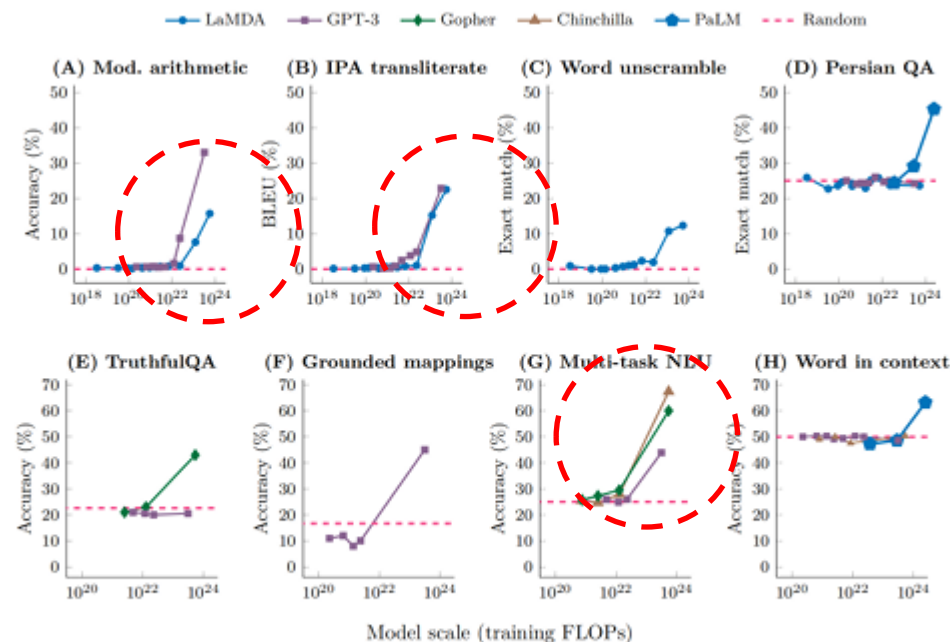


Figure 1: Emergent abilities of large language models. Model families display *sharp* and *unpredictable* increases in performance at specific tasks as scale increases. Source: Fig. 2 from [33].

- DE와 AI

- AI/ML 모델의 품질은 알고리즘보다 데이터 파이프라인의 품질에 크게 좌우 ("Garbage in, Garbage out")
- LLM, RAG 등 최신 AI 워크로드는 대용량·비정형·실시간 데이터 처리를 요구
- 정형 데이터 중심 ETL + 비정형, 벡터 등의 데이터엔지니어링
- AI 파이프라인 인프라의 역할

- DE와 AI

- 전통적 ETL
 - 테이블 형태 데이터, RDBMS 중심
- AI 파이프라인
 - 정형데이터 및 비정형 데이터(텍스트 및 이미지)
 - 벡터DB, GraphDB 등
 - 배치 및 실시간 임베딩을 통한 모델학습, RAG검색, 파인튜닝

- 데이터엔지니어링과 AI 파이프라인
 - 제조 QA
 - 센서/로그 데이터 → ChromaDB/Neo4j 기반 Graph RAG 파이프라인 구축
 - LLM 연동을 위한 데이터 전처리
 - 청킹(Chunking), 임베딩, 메타데이터 태깅 전략
 - MLOps와의 결합
 - 데이터 파이프라인 자동화 + 모델 재학습 트리거링(Continuous Training)

- 파이썬

- 1990년대 후반 Guido van Rossum이 개발 (네덜란드)
- 오픈소스 platform: General Public License (GPL)라이선스)
- Python의 특징: High-level / Interactive / 객체지향 / Scripting 언어
- NumPy, Pandas, PyTorch, scikit-learn, LangChain 등 데이터 분석·머신러닝·LLM 관련 라이브러리 대부분이 파이썬 기반
- Python native 방식 VS IDE
 - IDE
 - Spyder, pycharm, eclipse,...
 - Visual Studio Code**
 - Ipython 기반 Jupyter Notebook (ipykernel)
 - Colab

- 파이썬

- IDE 이용 시 Kernel

- 파이썬 코드를 해석하고 실행해서 결과를 제시해주는 엔진

- ① VS코드에 파이썬 코드 입력

- ② 해당 코드를 실행할 수 있는 파이썬 커널로 전달

- 커널: 사용PC 에 설치한 파이썬 또는 Colab에서 제공하는 파이썬
 - 로컬의 경우, 기본 파이썬과 가상환경에서의 파이썬 중 선택

- ③ 코드 실행 및 결과

- ④ 커널에서 실행 결과가 VS코드로 전달

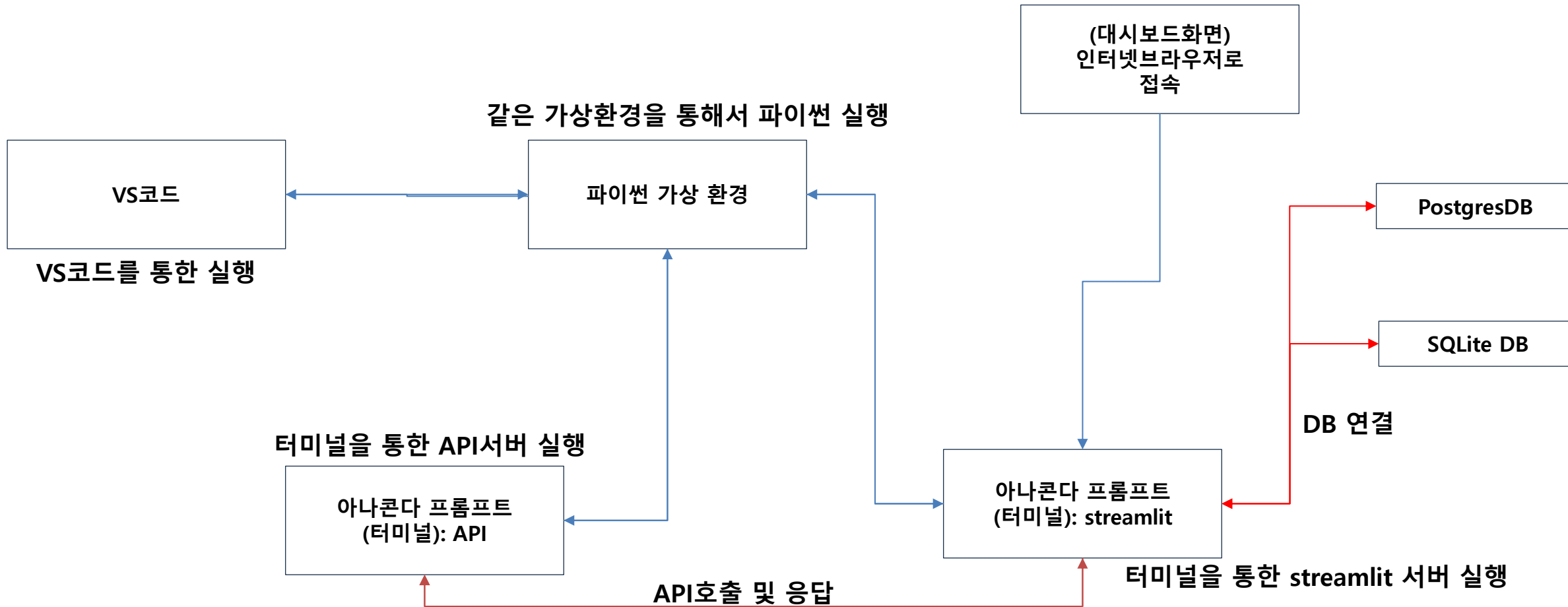
- ⑤ 결과 출력

- 파이썬

- VS코드 이용 시 Kernel 관련 오류
 - 윈도우 환경에서 미니콘다 설치, VS코드 설치, VS코드 내 Extension 설치 진행
 - 미니콘다의 conda 활용, 가상환경 설정
 - Python 3.13 사용
 - VS코드 실행 후 가상환경을 커널에서 선택
 - VS 코드 실행 시 관리 모드가 열리는 경우, 제공된 ipynb 파일 활용 허락
 - 가상환경 인식 못하는 경우, 미니콘다 설치 후 VS코드 재설치
 - 사내 보안으로 실행 전 대기시간 발생할 수 있음

I. DE 개요 및 실습환경 이해

노트북



• 파이썬 가상환경

- 가상환경이 필요한 이유
 - 프로젝트별로 서로 다른 파이썬 버전/패키지 버전이 필요한 경우 충돌 방지
 - 시스템 전역 환경 오염 방지, 재현 가능한 개발 환경 구성

#1. 기본

```
conda --version    # conda 버전
conda info         # conda 전체 설정
conda info --envs  # 환경 목록
conda env list     # 환경 목록 (동일)
conda list
conda list | findstr sklearn
```

#2. 가상환경 생성/삭제, 활성화/비활성화

```
conda create -n ds python=3.10
conda create -n ml python=3.9 numpy pandas scikit-learn
conda remove -n ds --all

conda activate ds
conda deactivate
```

#3. 패키지 설치 및 제거

```
conda install numpy
conda install pandas=1.5.3
conda remove pandas
conda install numpy pandas matplotlib
#conda install 후 pip설치, 반대 순서는 오류 가능
```

#4. 채널 관리

```
conda config --show channels
conda config --add channels conda-forge
conda config --set channel_priority strict
conda install -c conda-forge pyarrow
```

• 파이썬 가상환경

#5. 환경 export

저장

```
conda env export > environment.yml
```

복원

```
conda env create -f environment.yml
```

```
conda env create -n newenv -f environment.yml
```

#6. 예

1) 환경 생성

```
conda create -n test python=3.14
```

2) 활성화

```
conda activate test
```

3) 채널 설정

```
conda config --show channels
```

```
conda config --add channels conda-forge
```

```
conda config --set channel_priority strict
```

4) 패키지 설치

```
conda install numpy pandas scikit-learn seaborn
```

5) pip 보완

```
pip install mlxtend
```

II. 자료구조와 파이썬 문법

➤ 데이터 구조?

- 데이터를 일정한 기준에 의해 모아 놓은 것 (collection of data elements, structured in some way)

➤ List

- 여러 같은 type의 값을 하나의 객체로 모음
- 관련 함수: methods (object에 관련된 함수): len, max, min
- object.method(arguments) 방식으로 사용: lst.append(값), lst.count(값), lst.insert(위치, 값), lst.pop(), lst.reverse(), lst.remove(값), lst.sort()

II. 자료구조와 파이썬 문법

➤ List

- 숫치형 값 또는 문자열 등 여러 값의 모임
 - indexing: Sequence 내의 element에 대해 순서 별 index로 접근
 - slicing: 일정 범위 내의 element를 access하는 것
- 값의 추가: 같은 type의 값을 추가

II. 자료구조와 파이썬 문법

➤ Set

- list와 같은 값의 모임
- 값의 순서가 없음, 집합으로 이해
- { }로 생성

II. 자료구조와 파이썬 문법

➤ Tuple

- 다른 형태의 sequence, list와 유사, 0 index에서 시작하는 값
- (, ,)로 표현
- Immutable sequence: 일단 생성 후에는 값을 변경할 수 없음. 문자열과 유사
- sort, append, reverse 등의 method는 사용 불가

II. 자료구조와 파이썬 문법

➤ Dictionary

- 각 값마다 이름이 있는 값의 모음(A "bag" of values, each with its own label)
- 파이썬의 유용한 기능으로, key-value 구조를 지원
- Dictionary는 { }를 이용하여 생성 / 값으로는 key:value 쌍을 , 로 나열
- 빈 dictionary는 {}를 통해 생성

II. 자료구조와 파이썬 문법

➤ 조건문과 반복문

- 조건문: 조건의 참/거짓여부에 따라 문장을 각각 실행, if else, if elif else를 사용
- 반복문: 파라미터 값을 변경하면서 동일한 작업을 여러 번 실행, for 사용
- 조건문-bool을 사용하여 조건 충족 여부 판단
- block은 { }로 표현하지 않고 indentation으로 표현(참고: 한 레벨당 공백 4개)

II. 자료구조와 파이썬 문법

➤ 조건문과 반복문

```
var = 100
```

```
if var < 200:
```

if에 대한 block

```
    print("Expression value is less than 200")
```

```
    [ if var == 150: ] 하위 block
```

```
        print("Which is 150")
```

```
    elif var == 50:
```

```
        print("Which is 50")
```

```
    elif var < 50:
```

```
        print("Expression value is less than 50")
```

```
    else:
```

```
        print("Could not find true expression")
```

II. 자료구조와 파이썬 문법

➤ 조건문과 반복문

- 반복문-while과 for를 사용
- for: set/sequence(또는 iterable object(iterate가능 객체)의 각 값에 대해 Code block 수행

```
x =1
```

```
while x <=100:
```

```
    print(x)
```

```
    x+=1
```

```
words = ['this','is','a','wonder']
```

```
for word in words:
```

```
    print(word)
```

II. 자료구조와 파이썬 문법

➤ 조건문과 반복문의 들여쓰기

- 같은 level의 statement들은 같은 들여쓰기(indentation)값을 갖아야 함

```
if a==1:
```

```
    print(1)  
    print(0)
```

X

```
if a==1:
```

```
    print(1)  
    print(0)
```

O

II. 자료구조와 파이썬 문법

분기(Branch):

일방향적인 프로그램의 흐름이 어느 지점에서 2개 이상으로 나뉘게 됨

if : 분기(branch)를 위한 명령어

사용 방법:

if 조건식:

코드

코드

코드

└──────────┘

들여쓰기!

조건문의 중첩

if 반짝이는 것 == 동전:

if 반짝이는 것 == 500원짜리 동전:

줍기

if 반짝이는 것 < 500원짜리 동전:

그냥 가던 길 가기

II. 자료구조와 파이썬 문법

else 사용 방법:

if 조건식:

코드1

else:

코드2

True & True는 True
True & False는 False
False & True는 False
False & False는 False

True | True는 True
True | False는 True
False | True는 True
False | False는 False

여러 개의 조건식

if 조건식1 &(또는 |) 조건식2:

코드

if 조건식1 &(또는 |) 조건식2:

코드1

else

코드2

if 국어>90점 & 수학>90점:

피자

Else:

식빵

II. 자료구조와 파이썬 문법

for + if 사용

- ② 1에서 선택된 값이 임시 변수에 임시로 할당
- ① 시퀀스 자료구조에서 값이 하나씩 선택됨

for 임시변수 in 시퀀스자료구조:

- ③ 실행할 코드
임시변수에 조건문을
적용

시퀀스 자료
구조의
모든 값이
다 사용될때
까지 이 과
정이 반복

```
for i in sales:
```

```
    if i > 50:
```

들여쓰기!

```
        print(i)
```

또 들여쓰기!

II. 자료구조와 파이썬 문법

while 반복문

while 반복문은 조건식으로만 동작하며 반복할 코드 안에 조건식에 영향을 주는 변화식

사용방법

초기식

while 조건식:

반복할 코드

변화식

```
i = 0                # 초기식
while i < 10:         # while 조건식
    print('펏하 ' )   # 반복할 코드
    i = i + 1         # 변화식
```

For+리스트

```
a = [1,2,3,4]
result = []
for num in a:
    result.append(num*3)
print(result)
```



사용방법:
[표현식 for 항목 in 반복가능객체 if 조건]

```
result = [num * 3 for num in a]
print(result)
```

```
result = [num * 3 for num in a if num % 2 == 0]
print(result)
```

함수(Function)

반복적으로 계속 사용되는 일정한 작업을 수행하는 코드들의 모음

함수: 함수이름() 의 형식으로 사용

예: 함수 객체 확인하는 함수
globals()

함수의 입력과 출력

입력값없는 경우: 함수명()

입력값있는 경우: 함수명(입력값, 입력값2, ...)

출력값 있는 경우: 함수 내 return을 통해 반환

결과값 받는 변수 = 함수명()

II. 자료구조와 파이썬 문법

➤ 함수 정의

- def 라는 예약어와 함께 정의
- 함수의 사용은 함수명(입력값)의 형태

```
def Times(a,b):  
    return a*b
```

- 함수 객체 확인: globals()
- return 없으면 None 반환

```
def setValue( newValue):  
    x=newValue  
  
retval = setValue(10)  
print(retval)
```

II. 자료구조와 파이썬 문법

모듈, 패키지, 라이브러리

모듈: 특정 기능을 .py 파일 단위로 작성

패키지: 특정 기능과 관련된 여러 모듈을 묶음. 패키지는 모듈에 네임스페이스(namespace, 이름공간)를 제공

네임스페이스?

패키지1: 모듈1

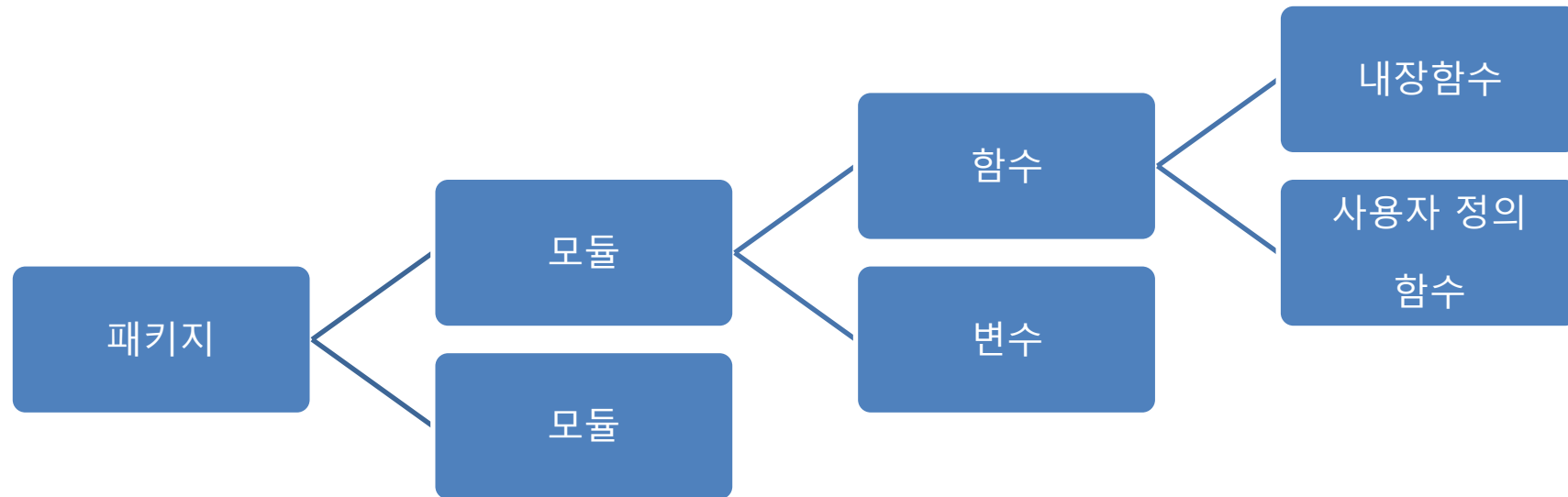
패키지2: 모듈1

패키지1의 모듈1을 쓰고 싶다면!

패키지1.모듈1

II. 자료구조와 파이썬 문법

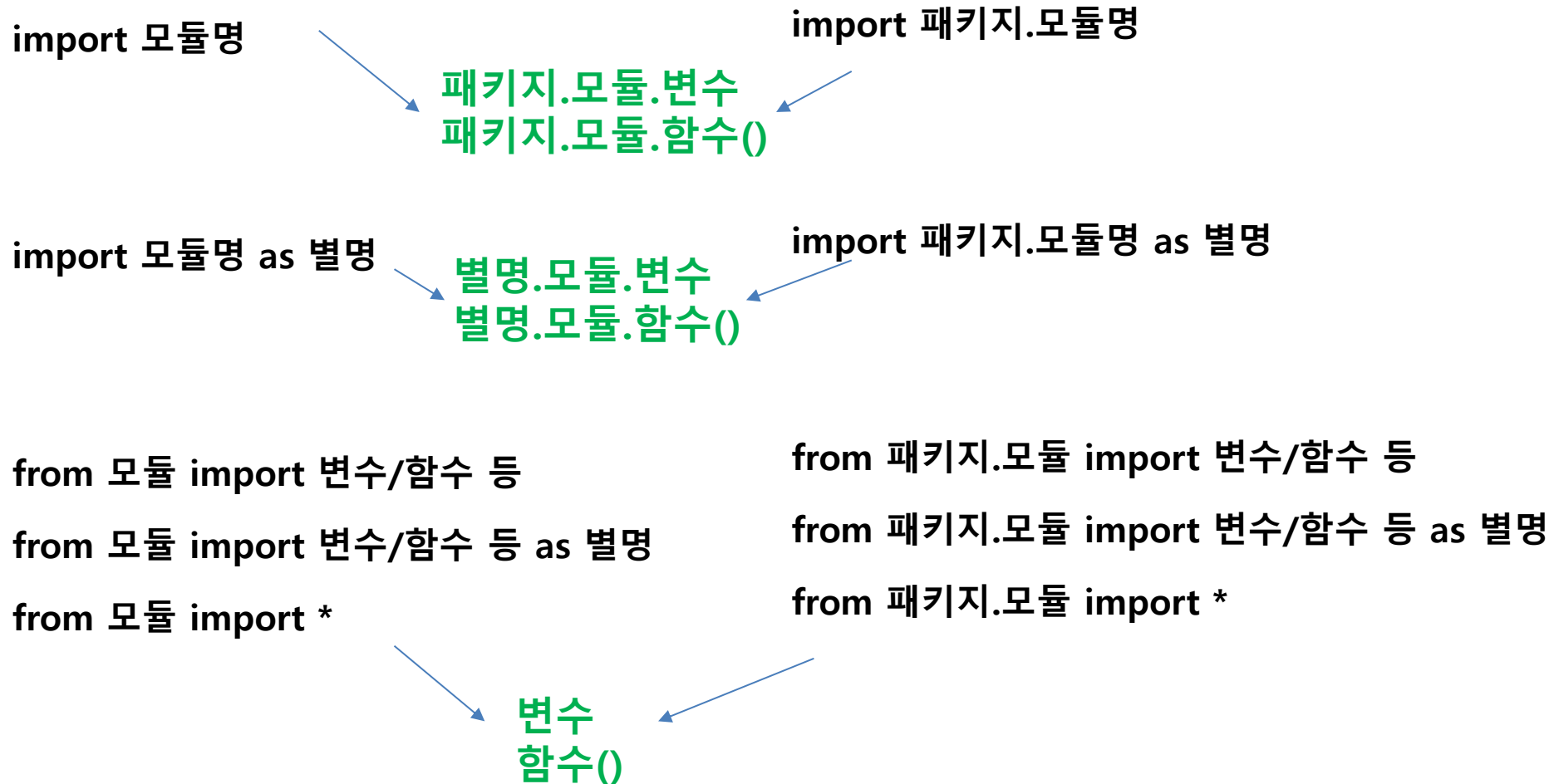
모듈, 패키지, 라이브러리



파이썬 표준 라이브러리: 파이썬에 기본으로 설치된 모듈과 패키지, 내장 함수를 통칭하여 파이썬 표준 라이브러리

II. 자료구조와 파이썬 문법

모듈이나 패키지를 불러오는 3가지 방법



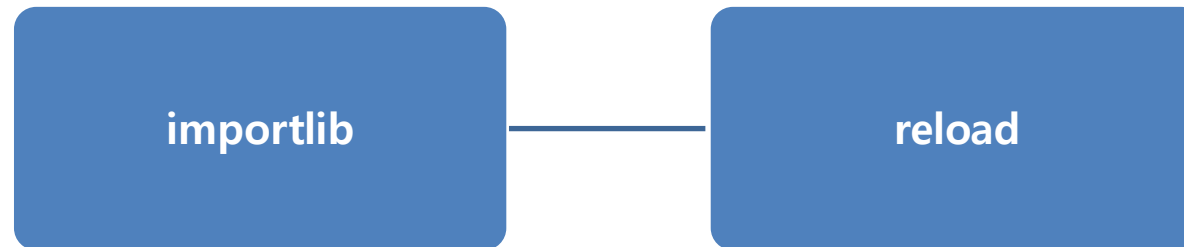
II. 자료구조와 파이썬 문법

모듈 해제와 다시 불러오기

`del 모듈`

`import importlib`

`importlib.reload(모듈)`



II. 자료구조와 파이썬 문법

모듈 설치하기

pip ?

파이썬 패키지 인덱스(Python Package Index, PyPI)의 패키지 관리 명령어

패키지 설치/삭제/업데이트 지원

쥬피터노트북에서 터미널-> **pip install 패키지 (또는 모듈) 이름**

예: **pip install mlxtend**

문자열(String)

따옴표, 또는 홑따옴표로 둘러싼 문자들의 집합

아무리 수치값이어도 문자열로 표현되면 더 이상 숫자가 아님!

"문자열은 어렵나요?"

'문자열은 사용하면 안되요'

'I can speak English'

'123'

II. 자료구조와 파이썬 문법

파일 열고 값 출력

내장함수 open 사용

이름=open(파일명, 파일열기모드)

*이름은 Open한 파일의 실제 값을 나타내는 역할을 함(객체)

파일열기모드

- 1) r: 읽기
- 2) w: 쓰기(기존 내용을 무시!)
- 3) a: 추가하기(기존 내용에 추가!)

파일 열고 출력의 3단계

1. 파일 열고 값 출력

```
file1=open( "file1.txt", 'r')
```

```
file1=open( "file1.txt", 'w') #해당 파일명이 없다면 파일이 생성됨
```

```
file1=open( "C:\\file1.txt", 'r')
```

2. 열린 파일에 값 쓰기

```
f.write(값)
```

3. 파일 열고 작업 후에는 파일을 닫기

```
file1.close()
```

파일 입력: 제공된 파일을 읽기

파일을 열은 후,

파일 객체.readline()

또는 파일 객체.readlines()를 이용

또는 파일 객체.read()를 이용

파일 읽기 3단계

1. 파일 열기 : r 모드
2. 파일 내용 읽어서 할당
3. 파일 닫기

II. 자료구조와 파이썬 문법

with 사용

```
with open( " 파일명 " , " w " ) as file1:
```

```
    file1.write( " 파일 닫기, 거참 번거롭구먼 허허")
```

II. 자료구조와 파이썬 문법

예외 처리: 오류 발생 시 적용

try except 사용

```
try:
    실행할 구문(들)
except:
    실행할 구문
```

```
try:
    실행할 구문(들)
except 발생 오류 코드1:
    실행할 구문(들)
except 발생 오류 코드2:
    실행할 구문(들)
except 발생 오류 코드3:
    실행할 구문(들)
```

오류 그냥 넘어가기: pass

array

▮ 1차원, 2차원, 3차원, ... 가능

▮ +, -, *, / 계산 지원

참고. arange 함수: 일정한 범위내 일련의 정수를 갖는 array 생성

```
import numpy as np
```

```
arr4 = np.arange(10)
```

#0부터 9까지 값을 갖는 1차원 array

```
arr5 = np.arange(3,10)
```

#3부터 9까지 값을 갖는 1차원 array

array의 다차원!

1차원 / 2차원 / 3차원

```
import numpy as np
```

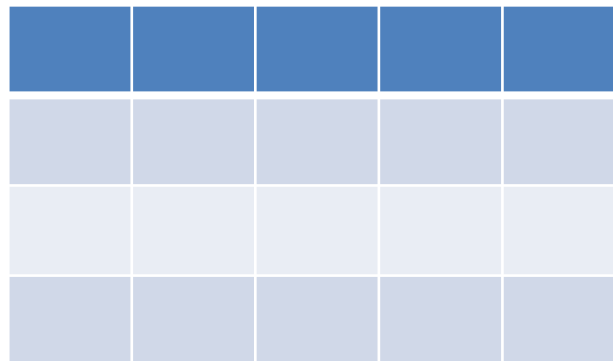
```
arr4 = np.array( [1,2,3] )
```

```
arr5 = np.array( [ [1,2,3], [4,5,6] ] )
```

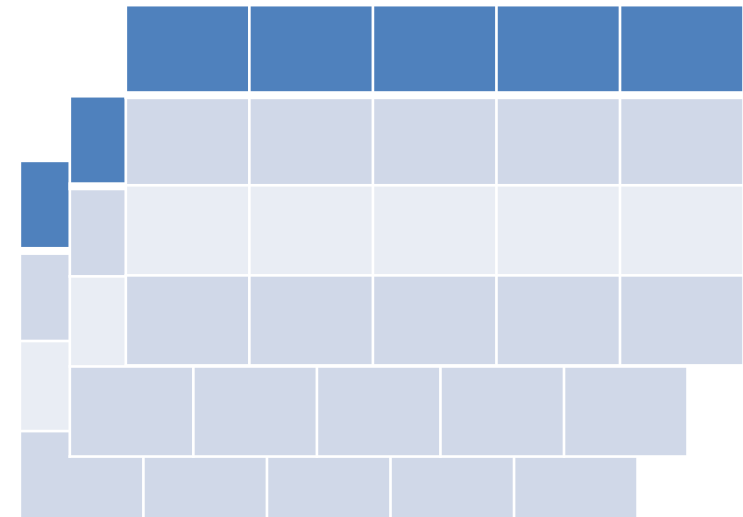
1차원



2차원



3차원



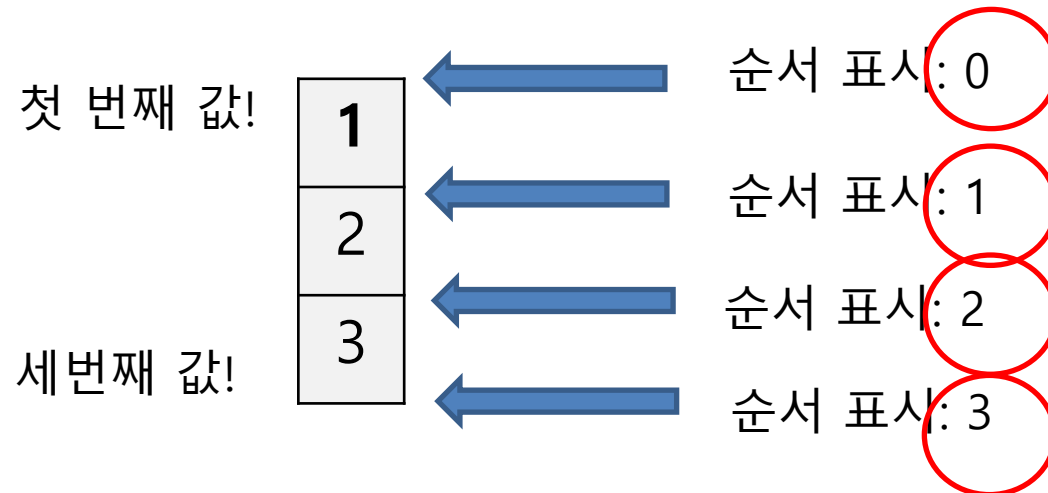
array 값들의 순서!

```
import numpy as np
```

```
arr4 = np.array( [1,2,3] )
```

```
arr4.tolist() #array의 list 변환 함수!
```

값의 순서가 있다!

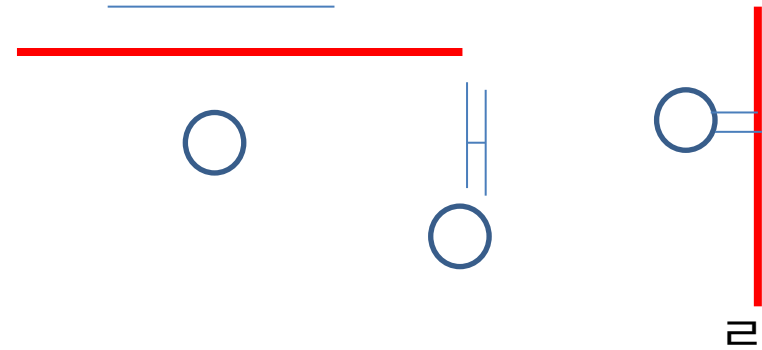


III. Numpy와 Pandas

array 값들의 순서!

```
import numpy as np
```

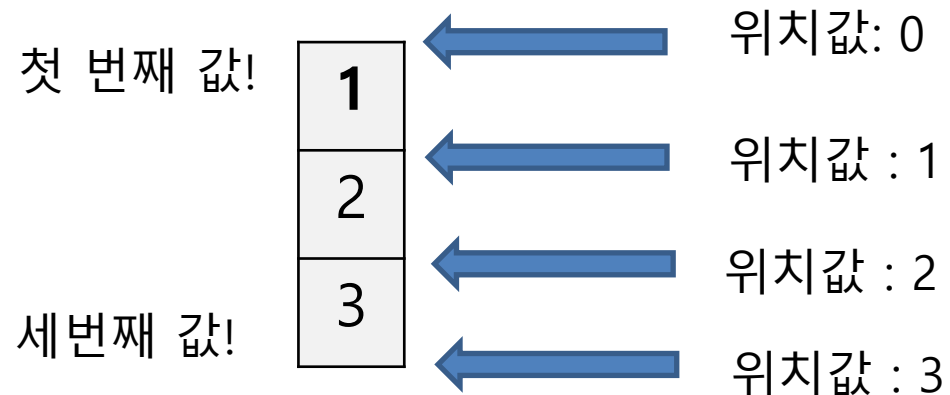
```
arr5 = np.array( [ [1,2,3], [4,5,6] ] )
```



	첫 번째 열!	두 번째 열!	세 번째 열!	
첫 번째 행!	1	2	3	← 순서 표시: 0
두 번째 행!	4	5	6	← 순서 표시: 1
				← 순서 표시: 2
	↑	↑	↑	↑
	순서 표시: 0	순서 표시: 1	순서 표시: 2	순서 표시: 3

1차원 array의 Indexing

```
import numpy as np  
arr4 = np.array( [1,2,3] )
```



[]

```
arr4의 2번째 값! arr4[ 1 ]  
1번째, 3번째 값! Arr4[ [0,2] ]
```

2차원 array의 Indexing

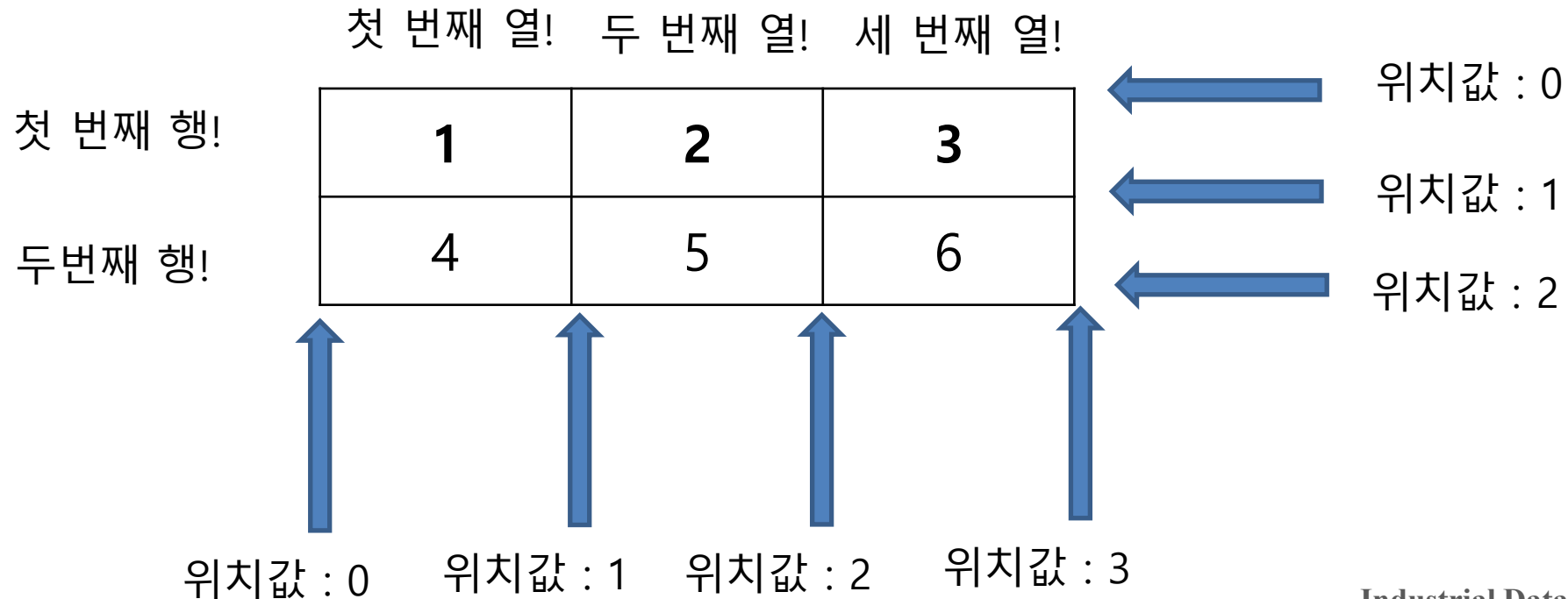
```
import numpy as np
```

```
arr5 = np.array( [ [1,2,3], [4,5,6] ] )
```

```
arr5[ 0 , :] #첫번째 행
```

```
arr5[ : , 1] #두번째 열
```

```
arr5[ 0 , 1] #첫번째 행, 두번째 열
```



다차원 array에서의 Slicing



Slicing에 사용되는 연산자!

[:]

Slicing에 : 의 사용!

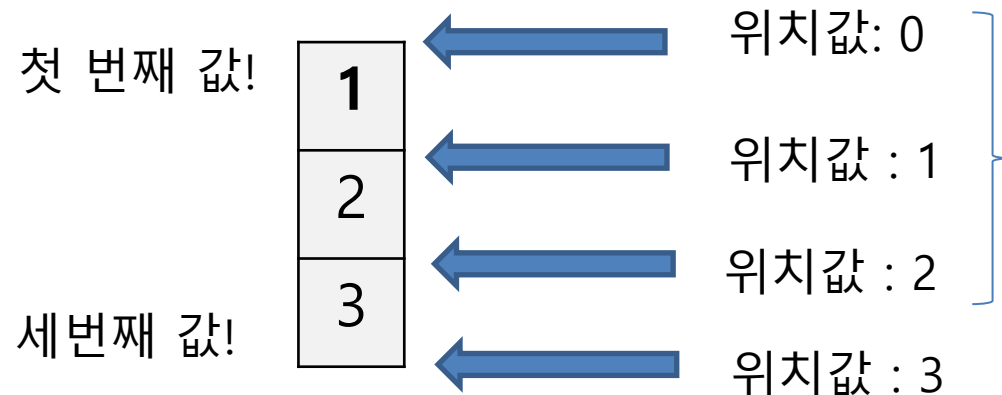
시작되는 위치:끝나는 위치

Slicing 사용 방식!

array이름[시작되는 위치값:끝나는 위치값]

1차원 array의 slicing

```
import numpy as np  
arr4 = np.array( [1,2,3] )
```



```
arr4의 1~2번째 값! arr4[ 0:2 ]  
arr4의 1~2번째 값! arr4[ 0:3 ]
```

2차원 array의 Slicing

```
import numpy as np
```

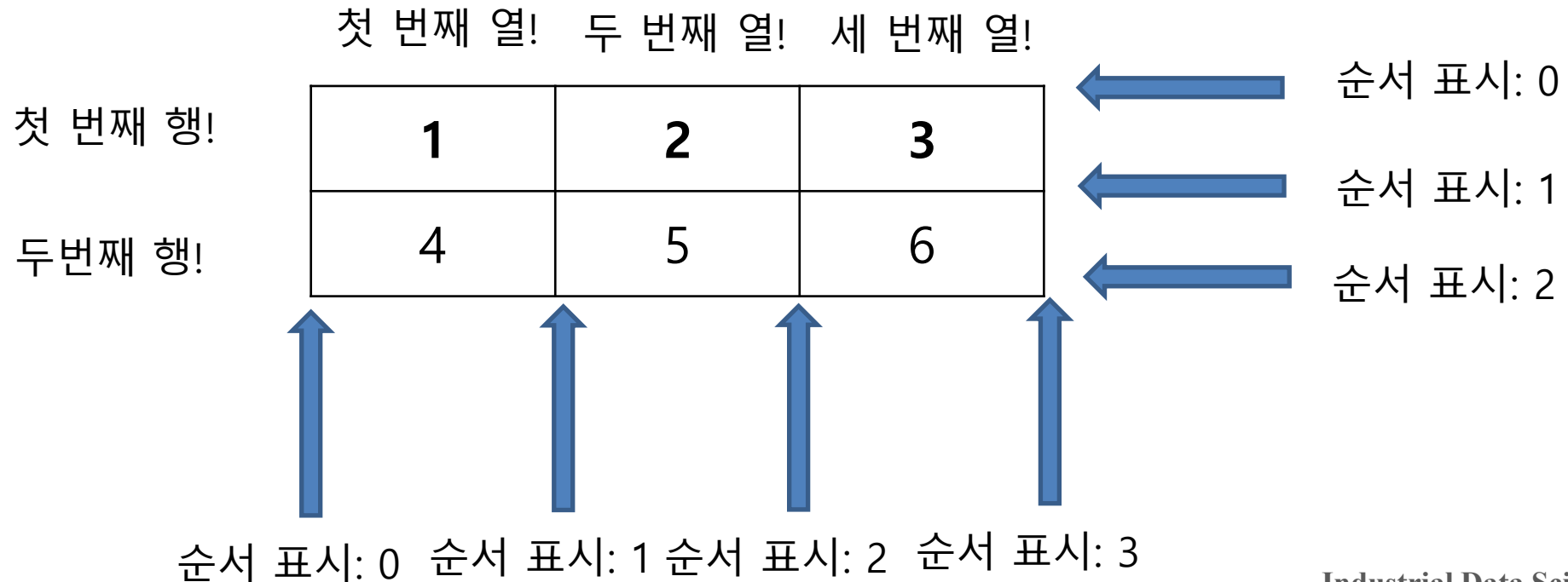
```
arr5 = np.array( [ [1,2,3], [4,5,6] ] )
```

```
arr5[ 0:1 , 1:3]
```

#1행, 2,3열

```
arr5[ 0:2 , 1:2]
```

#1,2행, 2열



array

array에 Scalar를 사칙연산이 가능

`arr4 * 3`

`arr5 + 1`

`arr4 - 3`

`arr5 / 2`

계산 결과를 새로운 array에 할당(assignment)

`arr6 = arr4*3`

array간 연산(1차원)

array 간 덧셈

`arr8+arr9`

array 간 뺄셈

`arr8-arr9`

array 간 곱셈

`arr8*arr9`

array 간 나눗셈

`arr8/arr9`

`arr8.size`

`arr8.shape`

`arr8.ndim`

주요 통계량 함수를 이용한 array 계산

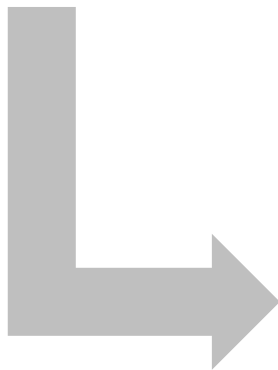
통계량? 주어진 자료의 특성을 나타내는 요약값

```
import numpy as np
```

```
arr12 = np.array( [3,5,2,4] )
```

```
arr13 = np.array( [3.1,5.2,4.5] )
```

```
arr14 = np.array( [-2,3,1,10] )
```



예: 평균, 편차, 최대, 최소값 등

평균: `np.mean( arr12 )`

표준편차: `np.std( arr12 )`

최대: `np.max( arr13 )`

최소: `np.min( arr13 )`

주요 통계량 함수를 이용한 array 계산

그 외에도 다양한 수학함수를 지원

sum, prod, abs, sqrt, exp, log 등등...

절대값: `np.abs( arr14 )`

제곱근: `np.sqrt( arr12 )`

지수: `np.exp( arr13 )`

로그: `np.log( arr13 )`

지수? Exponential!

로그? Log!

$$e = 2.71828182846$$

pi와 e를 다음처럼 미리 정의

$$pi = 3.14159265359$$

$$e = 2.71828182846$$

III. Numpy와 Pandas

➤ pandas

- 데이터 처리를 위한 최고의 패키지!
- 데이터 처리 기능을 풍부하게 제공
- 빠르고 직관적인 자료 구조!

➤ 특징

- DataFrame 제공
- 엑셀, CSV 등을 포함한 다양한 포맷 지원
- 데이터 정렬과 결측치 처리, 피벗 등 기능!
- 고급 인덱싱과 슬라이싱
- 데이터 합치기와 형태 변경 지원!



III. Numpy와 Pandas

➤ Pandas의 데이터 구조

➤ 데이터 구조

➤ Series (1D)

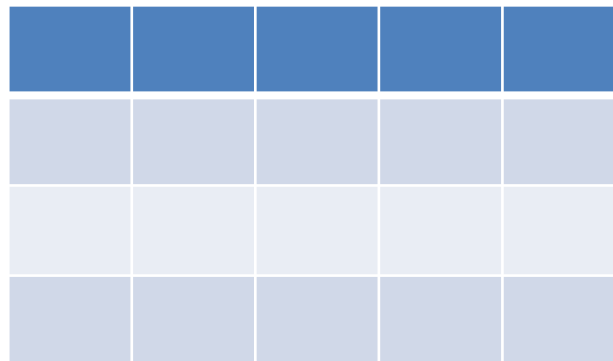
➤ DataFrame (2D)

➤ Panel (3D)

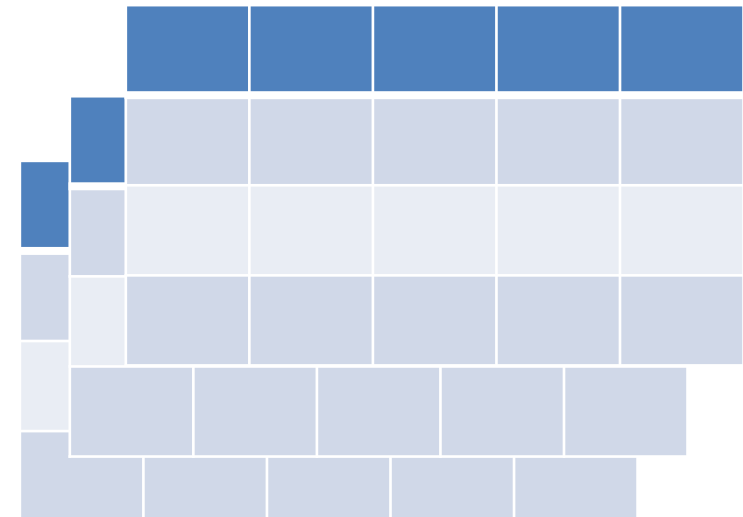
Series



DataFrame



Panel



III. Numpy와 Pandas

➤ Series

- 1차원
- Subclass of numpy.ndarray
- Index, Values를 갖음
- Index는 원하는 값을 넣을 수 있음-시계열

index		values
A	→	5
B	→	6
C	→	12
D	→	-5
E	→	6.7

III. Numpy와 Pandas

➤ DataFrame

- 행과 열이 있는 2차원
- 열마다 값의 타입이 다를 수 있음!
- 행, 열 모두 인덱스
- 크기가 바뀔 수 있음-행, 열 추가 및 삭제!

The diagram illustrates a DataFrame structure. At the top, a box labeled 'columns' contains the headers '국', '영', '수', and '출석'. Below this, a box labeled 'index' contains the row labels '갑', '을', '병', '정', and '무'. Arrows point from the column headers to the corresponding columns of data, and from the index labels to the corresponding rows of data. The data is organized into a grid of cells.

	columns	국	영	수	출석
갑	→	100	40	55	True
을	→	90	70	33	True
병	→	50	50	88	False
정	→	70	78	99	False
무	→	94	44	88	False

- **Series**
 - pandas에서 제공하는 1차원 자료 구조 , 연산, 인덱싱, 슬라이싱을 포함해 다양한 기능을 지원
- **Series의 인덱싱과 슬라이싱**
 - Series는 Index와 Value를 나누어 활용할 수 있으며, 이때 Index Label이나 Array를 이용한 인덱싱, 슬라이싱을 지원
- **Series 연산**
 - 수치 계산 및 벡터화 계산을 지원하며, Series 간 연산 시 같은 Index 값이 계산

III. Numpy와 Pandas

➤ Series

➤ 1차원

➤ Index, Values를 가짐

index	values
A	1
B	3
C	5
D	7
E	9

III. Numpy와 Pandas

➤ Series 생성하기

➤ 1차원 시퀀스 형태의 자료구조(예: 리스트 array)로 부터 생성 가능

➤ **Series = Value + Index**

➤ Index Label: Index에 원하는 값을 지정할 수 있음

```
import numpy as np
import pandas as pd

S1 = pd.Series( [1,2,3] )

S2 = pd.Series( np.array([1,2,3]))

S3 = pd.Series( [80,90,75], index=["가","나","다"] )
```

➤ **Series명.values**, 예를 들어 **S1.values**

➤ **Series명.index**, 예를 들어 **S3.index**

III. Numpy와 Pandas

➤ Series 인덱싱과 슬라이싱

- Index Label을 활용해서 원하는 범위의 값을 선택할 수 있음
 - Index가 지정 안 된 경우: 첫번째값이 0부터 시작하는 정수 index를 갖음
 - Index가 지정된 경우: 별도로 지정한 Index Label을 사용할 수 있으며, 첫번째값이 0부터 시작하는 정수 index도 사용할 수 있음

```
S4 = pd.Series( [1,2,3,4,5] )  
  
S5 = pd.Series( [1,2,3,4,5], index=["a","b","c","d","e"] )  
  
S4[0] # S4의 첫번째 값  
S4["a"] # 오류, S4에는 별도로 지정된 index 없음  
  
S5[0] # S5의 첫번째 값  
S5["a"] # S5의 첫번째 값  
  
S4[ 0:3 ] #S4의 1~4번째 값  
S5[ ["a", "c", "e"] ] #S5의 1,3,5번째 값 출력
```

III. Numpy와 Pandas

➤ Series 연산

➤ Series는 수치 계산을 지원

➤ $+, -, *, /$

➤ Series 간 연산은 같은 Index 값끼리 계산

```
S4 = pd.Series( [1,2,3,4,5], index=["a","c","d","b","e"] )
```

```
S5 = pd.Series( [1,2,3,4,5], index=["a","b","c","d","e"] )
```

S4+S5

S4

values	index
1	a
2	c
3	d
4	b
5	e

S5

index	values
a	1
b	2
c	3
d	4
e	5

- **DataFrame**
 - pandas에서 제공하는 2차원 자료 구조, 연산, 인덱싱, 슬라이싱을 포함해 다양한 기능을 지원
- **DataFrame의 인덱싱과 슬라이싱**
 - DataFrame은 행은 슬라이싱을, 열은 인덱싱을 지원할 수 있으며, 추가적인 함수를 통해 인덱싱과 슬라이싱을 자유롭게 사용 가능.
- **DataFrame 활용**
 - 수치 계산 및 벡터화 계산을 지원하며, 자료처리를 위한 다양한 함수를 활용할 수 있음

III. Numpy와 Pandas

➤ DataFrame의 특징!

- 2D table with rows and column labels
- column 별로 서로 다른 dtype을 가질 수 있음
- 행과 열 Index Label을 지원

The diagram illustrates a DataFrame structure with columns labeled 'foo', 'bar', 'baz', and 'qux' at the top. Below these, an 'index' column lists labels 'A', 'B', 'C', 'D', and 'E'. Arrows point from the column headers to the data cells. The data values are as follows:

	foo	bar	baz	qux
A	0	x	2.7	True
B	4	y	6	True
C	8	z	10	False
D	-12	w	NA	False
E	16	a	18	False

III. Numpy와 Pandas

➤ DataFrame의 더 많은 특징!

- 데이터의 reshaping 및 pivoting
- Label활용 슬라이싱과 인덱싱, 부분집합 구하기가 쉬움
- Group by 엔진 지원: Data Aggregation/ Data Transformation

➤ Data Alignment

- "Outer join"을 지원: 두 데이터프레임의 행 또는 열 Index를 기준으로 계산!

B	1		A	0	A	NA
C	2	+	B	1	B	2
D	3		C	2	C	4
E	4		D	3	D	6
		=			E	NA

III. Numpy와 Pandas

➤ DataFrame 생성하기

- 2차원 형태의 자료구조(예: 리스트의 리스트, 2차원 array)와 Dictionary(Value가 1차원 자료구조인 경우)로 부터 생성 가능

➤ DataFrame = Series + Series + Series

- 행과 열의 Index Label: Index에 원하는 값을 지정할 수 있음!

```
import numpy as np
import pandas as pd

df1 = pd.DataFrame( [[1,2,3], [1,2,3]], columns=['a','b'], index=['r1','r2','r3'] )
df2 = pd.Series( np.array([[1,2,3], [1,2,3]]) , columns=['a','b'],
index=['r1','r2','r3'])
raw_data = {'col0': [1, 2, 3, 4],
            'col1': [10, 20, 30, 40],
            'col2': [100, 200, 300, 400]}
df3 = pd.Series( raw_data )
```

- DF명.values, 예를 들어 df1.values
- DF명.index, 예를 들어 df1.index
- DF명.columns, 예를 들어 df1.columns

III. Numpy와 Pandas

- DataFrame의 인덱싱과 슬라이싱

- 인덱싱: 열 이름을 지정, 복수인 경우 리스트로 묶어서 지정
- 슬라이싱: 선택을 원하는 행을 슬라이싱 / 단 행 단위에 인덱싱 사용 불가

```
#행
df1[ 0 ] #오류
df1[ 0:1 ]

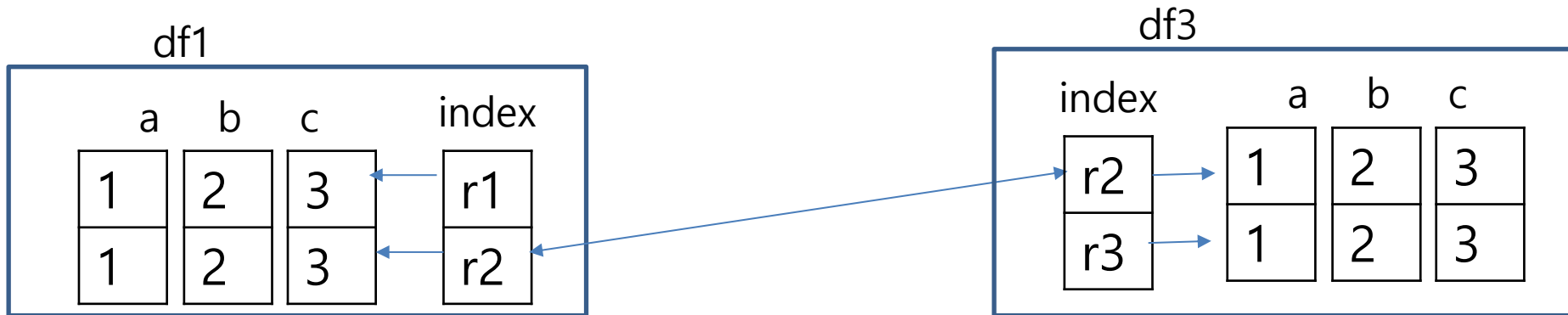
#열
df1['a']
df1[['a','c']]
```

III. Numpy와 Pandas

➤ DataFrame 활용

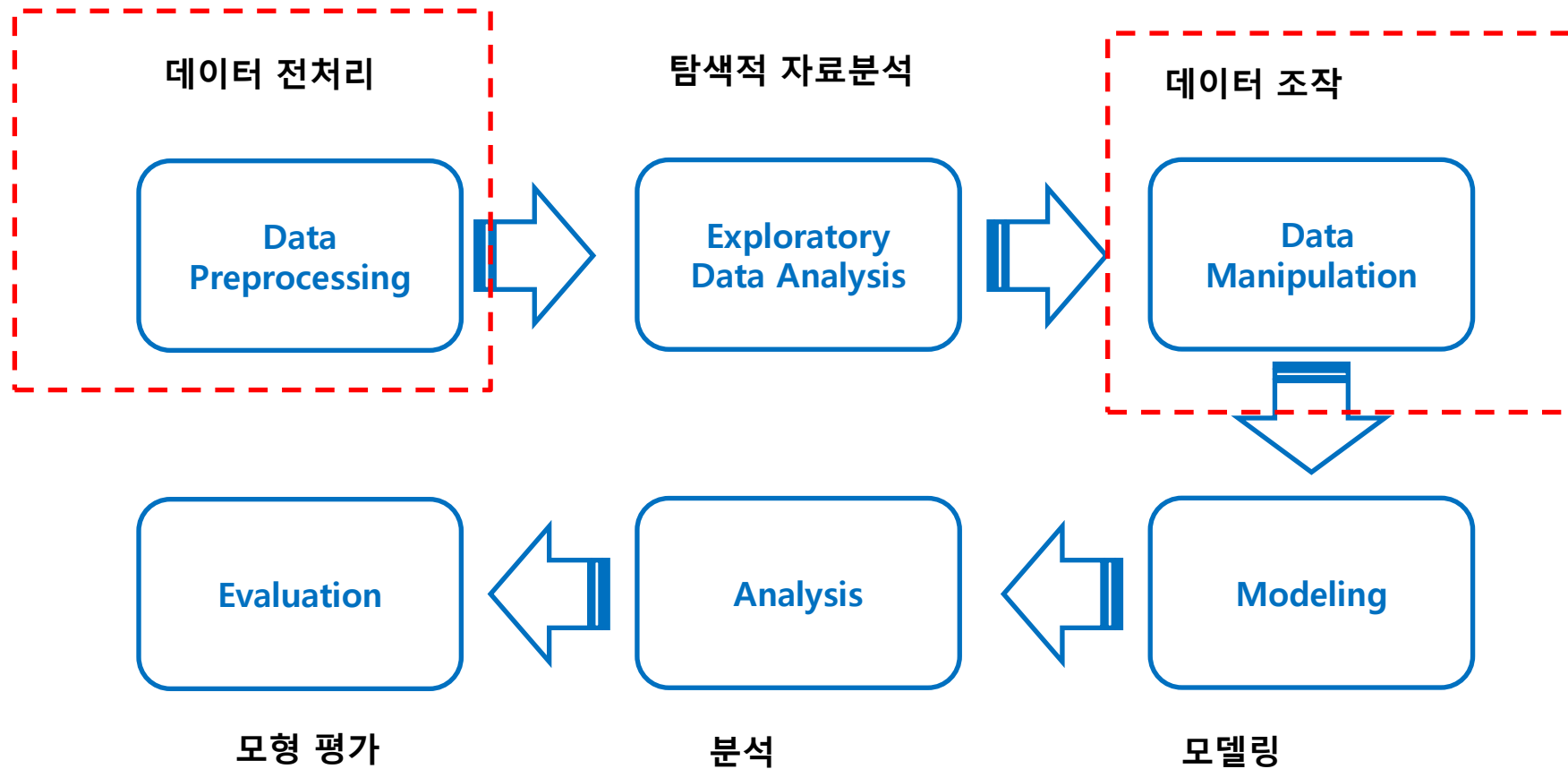
- .head(), .describe() 등의 유용한 기능 활용
- 두 데이터프레임의 수치 계산은 같은 인덱스의 해당되는 컬럼끼리 계산!
 - $+, -, *, /$

```
df3 = pd.DataFrame( [[1,2,3], [1,2,3]], columns=['a','b','c'], index=['r2','r3'] )  
df1+df3
```



IV. 데이터 전처리

➤ 개요



➤ Sklearn의 주요 모듈

- impute : missing value를 위한 transformer
- model_selection: Cross-validation(evaluating estimator performance), Tuning the hyper-parameters of an estimator, Learning curve 등에 사용
- Preprocessing: scaling, centering, normalization, binarization methods 등을 제공

➤ **imputing**

- Missing data 포함한 행 삭제
- Missing data의 대체 - mean / median imputation
- Missing data의 대체 - mode / frequent category imputation
- Missing data의 대체 - Random Sample Imputation

IV. 데이터 전처리

➤ partitioning

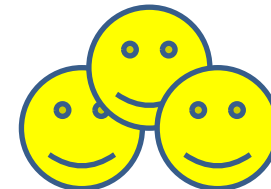
- 모델을 구축하고 모델의 성능을 평가하기 위해 주어진 데이터를 훈련 데이터와 테스트 데이터로 나누는 것을 의미. 이때 훈련 데이터와 테스트 데이터는 랜덤하게 선택되며, 서로 중복되지 않음.

데이터

이메일	대박	보고	미팅	견적서
정상	0	2	2	0
정상	0	2	3	0
정상	0	3	2	1
스팸	5	0	0	0



결과



IV. 데이터 전처리

➤ partitioning

➤ 예

데이터를 알뜰하게 사용!

이메일	대박	보고	미팅	견적서
정상	0	2	2	0
정상	0	2	3	0
정상	0	3	2	1
스팸	5	0	0	0

모형 구축에 사용!

- Train data(훈련데이터): 주어진 데이터 중 일부를 선택하여, 중 모형을 구축하는데 사용되는 데이터
- Test data(테스트데이터): 주어진 데이터에서 훈련데이터를 제외한 남은 데이터로, 훈련 데이터로 구축된 모형의 성능을 평가하는데 사용

모형 평가에 사용!

IV. 데이터 전처리

훈련 및 테스트 데이터의 선택은 ?

➤ partitioning

➤ 예

순서대로

모델 구축에 사용!

모델 평가에 사용!

이메일	대박	보고	미팅	견적서
정상	0	2	2	0
정상	0	2	3	0
정상	0	3	2	1
스팸	5	0	0	0

랜덤하게

모델 구축에 사용!

모델 평가에 사용!

➤ Scaling

- 데이터 정규화: 모델링 과정에서 모든 데이터가 동일한 정도의 크기로 학습에 반영되도록 해주기 위해 값을 변환하는 과정으로, Min-Max와 Standardization이 많이 사용됨.
- Min-Max 정규화: 정규화 대상 각 값들에서 전체 값의 최소값을 빼고, 그 값을 최대값과 최소값의 차이로 나눠주는 방식으로 값을 변환
- Standardization: Z-Score 정규화라고도 하며, 정규화 대상 각 값들에서 평균을 빼고 그 값을 표준편차로 나누는 방식으로 값을 변환.

IV. 데이터 전처리



예

포트폴리오A 100,000,000원/주

500,000,000원/주

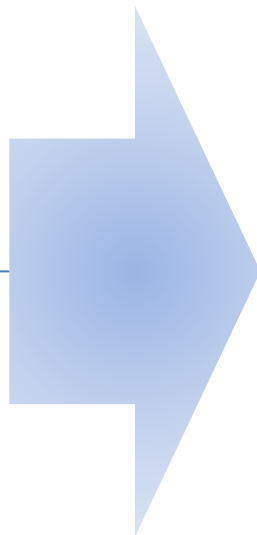
1,000,000,000원/주

포트폴리오B

10원/주

5원/주

100원/주



숫자들이 너무 크다!



데이터 정규화



숫자들이 너무 작다!

상대적
크기
차이
유지!

MinMax 정규화:
$$\frac{\text{각 값} - \text{최소값}}{\text{최대값} - \text{최소값}}$$

Standardization:
$$\frac{\text{각 값} - \text{평균}}{\text{표준 편차}}$$

